

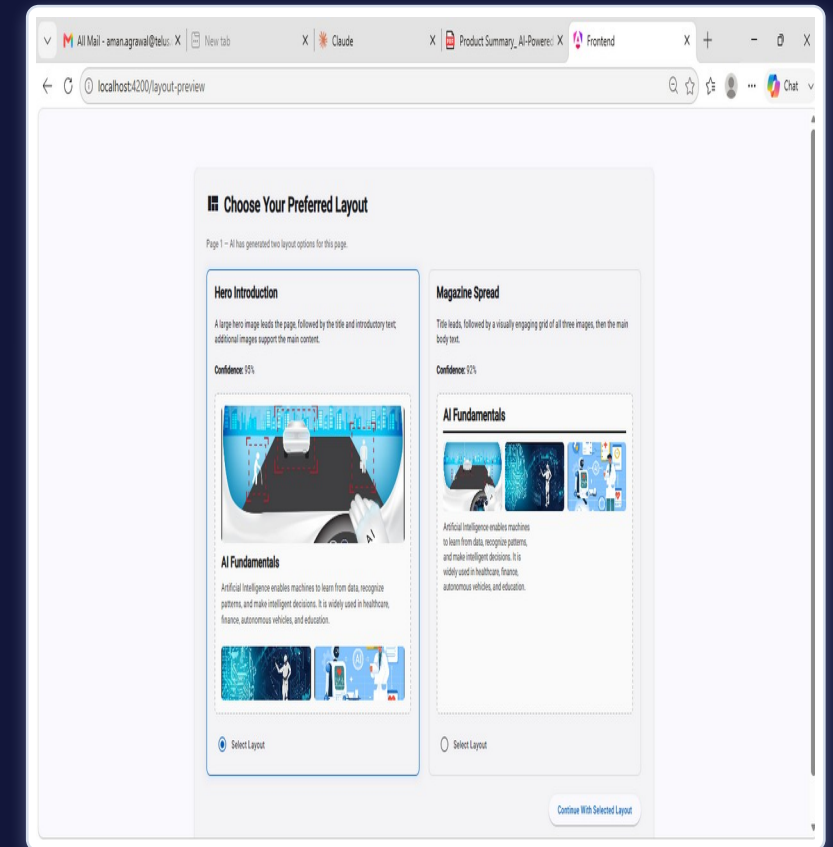
CODE WALKTHROUGH

AI-Powered Visual Book Generator

A source-code deep dive: FastAPI, the LLM layout engine, and Angular rendering.

Aman Agrawal

github.com/Aman007Agg/ai-powered-visual-book-generator



Agenda

1

Folder structure

How the codebase is organised

2

Tech choices

Angular · FastAPI · OpenAI — and why

3

The contract

The JSON the LLM returns

4

Backend

FastAPI app, endpoint, schemas

5

The AI core

Prompt + OpenAI service = why it's AI-powered

6

Frontend

API, state, models, the layout renderer

Folder Structure

```
ai-powered-visualbook-generator
|-- backend
|   |-- app
|   |   |-- api/      FastAPI routers (health, layout)
|   |   |-- core/     exceptions
|   |   |-- prompts/  system + user prompt builders
|   |   |-- schemas/  Pydantic request/response models
|   |   |-- services/  OpenAIService (orchestration + errors)
|   |   |-- main.py    app + CORS + router registration
|   |-- tests/         pytest (schemas, prompt, endpoint)
|   |-- requirements.txt
|-- frontend
|   |-- src/app
|   |   |-- core/services/ http (APIService) + state (BookState)
|   |   |-- features/     routed screens
|   |   |-- shared/
|   |       |-- components/renderers/ LayoutRenderer + Hero/SplitMagazine
|   |       |-- models/      typed contracts
|   |       |-- pipes/       ImageRefPipe
|   |       |-- utils/       groupSections()
|-- docs /README.md
```

What this does

- Clear separation: backend (AI + API) vs frontend (UI + rendering).
- Feature-first Angular structure; shared renderers, models, pipes, utils.
- Backend split by responsibility: api, schemas, prompts, services.

Why This Stack

Angular

Typed, component-driven SPA. Signals give simple, leak-free reactive state; owns all rendering.

FastAPI

Async, high-performance API. Pydantic validation makes the AI I/O a typed, safe contract.

OpenAI

Strong reasoning + JSON-object output we can validate and render deterministically.

What the LLM Returns (the contract)

```
{
  "page_summary": "Short summary of the page",
  "layout_options": [
    {
      "id": "layout_1",
      "layout_type": "hero",      //hero | split | magazine
      "layout_name": "Hero Image",
      "description": "Large hero image followed by text.",
      "confidence": 95,          //model self-rated 0-100
      "sections": [
        { "type": "title", "text": "Artificial Intelligence" },
        { "type": "image", "image_reference": 0 },
        { "type": "paragraph", "text": "AI enables machines..." }
      ]
    },
    { "id": "layout_2", "layout_type": "split", "sections": [ ... ] }
  ]
}
```

What this does

- The LLM outputs SEMANTICS as JSON — never HTML/CSS.
- Angular maps this JSON to a visual layout.
- This clean contract = security, determinism, testability.

App Setup (main.py)

```
app = FastAPI(
    title= "AIPowered VisualBook Generator API",
    version= "1.0.0",
)

# Allow the Angular app (localhost:4200) to call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins= ["http://localhost:4200"],
    allow_credentials= True,
    allow_methods= ["*"],
    allow_headers= ["*"],
)

app.include_router(health_router) # GET /
app.include_router(layout_router) # POST /api/layout/generate
```

What this does

- FastAPI is the bridge between Angular and OpenAI.
- CORS lets the browser call the API cross-origin.
- Routers register the endpoints; app stays stateless.

The Endpoint (api/layout.py)

```
router = APIRouter(prefix="/api/layout", tags=["Layout"])

@router.post("/generate")
def generate_layout(request: GenerateLayoutRequest):
    # FastAPI has ALREADY validated `request` against the
    # Pydantic schema before this line runs.
    return OpenAIService.generate_layout(request)
```

What this does

- Thin router: it only receives a validated request and delegates.
- Type hint = automatic request validation (422 on bad input).
- All logic lives in OpenAIService — easy to test.

Typed Contracts (Pydantic)

```

class GenerateLayoutRequest(BaseModel):
    title: str | None = None # optional context
    objective: str | None = None # optional context
    page_text: str = Field(..., min_length=1) # required
    in_ages: list[PageInAge] = Field(default_factory=list)

class LayoutSection(BaseModel):
    type: SectionType # title | paragraph | image
    text: str | None = None
    image_reference: int | None = Field(default=None, ge=0)

class LayoutOption(BaseModel):
    id: str
    layout_type: str # hero | split | magazine
    layout_name: str
    description: str
    confidence: int = Field(ge=0, le=100)
    sections: list[LayoutSection]

class GenerateLayoutResponse(BaseModel):
    page_summary: str
    layout_options: list[LayoutOption]

```

What this does

- One schema validates the request; one validates the AI response.
- Bounds (min_length, ge/le) reject bad data automatically.
- This is the safety net around a non-deterministic model.

The Prompt = the AI's Instructions (abridged)

```
SYSTEM_PROMPT = """
You are an expert editorial designer specializing in book layouts.
Generate EXACTLY TWO different layout recommendations.

Your task is NOT to generate HTML, CSS, or coordinates.
Decide ONLY: which layout, section order, which image per section.

CONSTRAINTS:
- "layout_type" MUST be one of: "hero", "split", "magazine".
- The TWO options MUST use TWO DIFFERENT layout_type values.
- "confidence" is an integer 0-100.
- "image_reference" is 0-based and < number of images.

Return ONLY valid JSON (no markdown, no explanations).
"""
```

What this does

- Prompt engineering: role, goal, hard constraints, output schema.
- Forces two DIFFERENT layouts so the choice is meaningful.
- Explicitly forbids HTML/CSS — the model only reasons.

Dynamic User Prompt (build_user_prompt)

```
def build_user_prompt(title, objective, page_text,
                      in_age_count) -> str:
    return f"""
    Book Title:    {title or "Not Provided"}
    Book Objective: {objective or "Not Provided"}
    Uploaded Images: {in_age_count}

    Page Content:
    {page_text}

    Generate EXACTLY TWO layout options using two DIFFERENT
    layout_type values. Never use an in_age_reference >= {in_age_count}.
    """
```

What this does

- Injects this page's real content + context into the prompt.
- Passes the image COUNT (vision is scoped for the MVP).
- Repeats the key constraints right next to the data.

Where the LLM Works (openai_service.py)

```

response = client.chat.completions.create(
    model=MODEL_NAME,          # gpt-4.1
    response_format={"type": "json_object"}, # force JSON
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.5,
)

data = json.loads(response.choices[0].message.content)
validated = GenerateLayoutResponse.model_validate(data) # Pydantic
return validated.model_dump()

# except AuthenticationError -> 401, RateLimitError -> 429,
# BadRequestError(context) -> 400, others -> 500 (typed errors)

```

What this does

- THIS is the AI: GPT reasons about the page and returns a layout.
- json_object mode + Pydantic = safe, structured output.
- Typed error mapping turns OpenAI failures into clean HTTP codes.

Calling the API (ApiService)

```
@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  private apiUrl = environment.apiUrl + '/layout/generate';

  generateLayouts(req: GenerateLayoutRequest)
    : Observable<GenerateLayoutResponse> {
    return this.http
      .post<GenerateLayoutResponse>(this.apiUrl, req)
      .pipe(
        retry({ count: 1, delay: 1000 }), // transient failures
        catchError(err => throwError(() => err)),
      );
  }
}
```

What this does

- The one place Angular talks to the backend (typed HttpClient).
- RxJS is used here — the natural fit for an async HTTP call.
- Retries once before surfacing an error to the UI.

Single Source of Truth (BookStateService)

```
@Injectable({ providedIn: 'root' })
export class BookStateService {
  private _pages = signal<BookPage[]> ([]);
  private _draftResponse =
    signal<GenerateLayoutResponse | undefined> (undefined);

  readonly pages = this._pages.asReadonly();
  readonly pageCount = computed(() => this._pages().length);

  commitCurrentPage(selectedLayoutId: string): void {
    const res = this._draftResponse();
    const layout = res?.layout_options
      .find(o => o.id === selectedLayoutId);
    if (!res || !layout) return;
    this._pages.update(p => [...p, {
      pageNumber: p.length + 1,
      selectedLayout: layout,
      /* + pageText, images, pageSummary */
    }]);
  }
}
```

What this does

- Signals store: reactive, glitch-free, zoneless-friendly.
- `commitCurrentPage()` turns the chosen option into a book page.
- Private writable signals; public read-only — safe encapsulation.

Typed Models Angular Consumes

```
export type SectionType = 'title' | 'paragraph' | 'image';

export interface LayoutSection {
  type: SectionType;
  text?: string;
  image_reference?: number;
}

export interface LayoutOption {
  id: string;
  layout_type: string; //hero | split | magazine
  layout_name: string;
  description: string;
  confidence: number;
  sections: LayoutSection[];
}
```

What this does

- Front-end mirrors the backend JSON contract as TypeScript types.
- Strong typing catches mistakes at compile time.
- Same shape end-to-end: Pydantic <-> TypeScript interfaces.

How Angular Renders the Layout (LayoutRenderer)

```
// layout-renderer.ts
export class LayoutRenderer {
  readonly layout = input.required<LayoutOption> ();
  readonly images = input<PageImage[]> ([]);
  readonly layoutType = computed(() =>
    (this.layout().layout_type ?? '').toLowerCase().trim ());
}

// layout-renderer.html (Strategy pattern via @ switch)
@ switch (layoutType()) {
  @ case ('split') { <app-split-layout .../> }
  @ case ('magazine') { <app-magazine-layout .../> }
  @ default { <app-hero-layout .../> } //+ fallback
}
```

What this does

- Reads layout_type and picks a component at runtime (Strategy).
- Adding a layout = new component + one branch (open/closed).
- Unknown type falls back to Hero — the UI never breaks.

JSON -> Pixels: helpers

```
// section-grouping.ts (simplified)
export function groupSections(sections) {
  const g = { title: '', paragraphs: [], imageRefs: [] };
  for (const s of sections ?? []) {
    if (s.type === 'title' && s.text) g.title += s.text;
    if (s.type === 'paragraph' && s.text) g.paragraphs.push(s.text);
    if (s.type === 'image' && s.image_reference !== null)
      g.imageRefs.push(s.image_reference);
  }
  return g;
}

// image-ref-pipe.ts — resolve index -> base64
transform (ref, images) {
  return (ref === null || !images) ? '' : images[ref]?.base64 ?? '';
}
```

What this does

- Each layout groups sections by role, then arranges them its own way.
- The pipe maps the AI's image_reference index to the real image.
- HeroLayout/Split/Magazine reuse these — DRY + testable.

Real Problems Solved

Two options looked identical

Renderer ignored layout_type -> built a Strategy renderer + prompt forces two different types.

Image preview lagged one click

Zoneless change detection missed the async FileReader -> moved to Signals.

Hero image cropped / gaps

Tuned object-fit (cover vs contain) + sized images to their aspect.

Single page -> full book

Introduced a Signals BookStateService (Book / BookPage model).

Unsafe LLM output

json_object + Pydantic validation; bad output fails, never reaches UI.

'Vision' not wired

Caught in QA; scoped + documented with a one-line upgrade path.

End-to-End, in One Line

Angular sends text + images -> FastAPI validates & prompts -> OpenAI returns JSON -> Pydantic validates -> Angular renders the chosen layout.

Why it's an AI-powered book generator: the layout decisions themselves are made by the LLM reasoning over each page — not by fixed templates.

Aman Agrawal github.com/Aman007Agg/ai-powered-visual-book-generator